

Inside a Coding-Agent Harness

A repository repair, the execution loop, and the evidence for a change

CMSC 848N Fall 2026

W7-L1 October 15 Furong Huang

The coding-agent harness

The harness builds model inputs, executes actions, returns observations, and decides when execution stops.

We will follow one recorded repair and distinguish the model's decisions from the tools' behavior.

We will inspect the implementation and test a choice: what should happen when an edit fails lint?

Next lectures: training trajectories, then automatic harness optimization.

The task: read floating-point pixel data

pydicom issue #1457 reports a failure when reading FloatPixelData.

```
ds.BitsAllocated = 32
ds.SamplesPerPixel = 1
ds.Rows = 5
ds.Columns = 5
ds.PhotometricInterpretation = 'MONOCHROME2'

pixel_array = np.zeros((5, 5), dtype=np.float32)
ds.FloatPixelData = pixel_array.flatten().tobytes()

result = np.array_equal(ds.pixel_array, pixel_array)
```

Expected here: reading the bytes recovers the same 5×5 array.

The reproduction identifies a required attribute

Recorded action 3

```
$ python reproduce_bug.py
```

```
.../pixel_data_handlers/numpy_handler.py, line 293  
    raise AttributeError(...)
```

```
AttributeError: Unable to convert the pixel data  
as the following required elements are missing  
from the dataset: PixelRepresentation
```

The traceback names the handler and the missing attribute.

The traceback identifies which file to open

Recorded action 4: find_file "numpy_handler.py"

```
pydicom/overlays/numpy_handler.py  
pydicom/pixel_data_handlers/numpy_handler.py  
pydicom/waveforms/numpy_handler.py
```

```
open pydicom/pixel_data_handlers/numpy_handler.py 293
```

The viewer supplies line numbers and a bounded window around the failure.

The handler requires integer-pixel metadata

PixelRepresentation marks signed or unsigned integer pixels. Floating-point pixels should not require it.

```
287 required_elements = [  
288     'BitsAllocated', 'Rows', 'Columns', 'PixelRepresentation',  
289     'SamplesPerPixel', 'PhotometricInterpretation'  
290 ]  
291 missing = [elem for elem in required_elements if elem not in ds]  
292 if missing:  
293     raise AttributeError(  
294         'Unable to convert the pixel data ...'  
295         + ', '.join(missing)  
296     )
```

Attempt 2 replaces lines 287–295 inclusive and supplies its own closing parenthesis. Which original line remains?

Why the second edit fails

Recorded second edit attempt: edit 287:295

Proposed replacement ends with

```
raise AttributeError(  
    "Unable to convert ..."  
    + ", ".join(missing)  
)
```

E999 SyntaxError: unmatched ')

Recorded candidate file ends with

```
294 raise AttributeError(  
295     "Unable to convert ..."  
296     + ", ".join(missing)  
297 )  
298 )
```

Original line 296 survives as candidate line 298.

Three rejected edits, then an accepted patch

The tool writes the proposed edit, runs its checks, then restores the backup on failure.

Attempt	Replacement range	Tool feedback
1	287:295	unmatched ']'
2	287:295	unmatched ')
3	287:295	unmatched ')' (same edit repeated)
4	287:296	File updated

After each rejection, the next model turn sees the restored file.

The accepted patch changes one requirement

Semantic core of the submitted patch, reformatted for readability

```
required_elements = [  
    'BitsAllocated', 'Rows', 'Columns',  
    'SamplesPerPixel', 'PhotometricInterpretation'  
]  
if 'PixelData' in ds:  
    required_elements.append('PixelRepresentation')
```

FloatPixelData bypasses this added condition. Integer PixelData still triggers it.

The agent reruns its reproduction and submits

Recorded actions 10–12

```
$ python reproduce_bug.py
```

```
Script completed successfully, no errors. Result: True
```

```
$ rm reproduce_bug.py
```

```
$ submit
```

Recorded status: submitted. The trace contains no test-suite run.

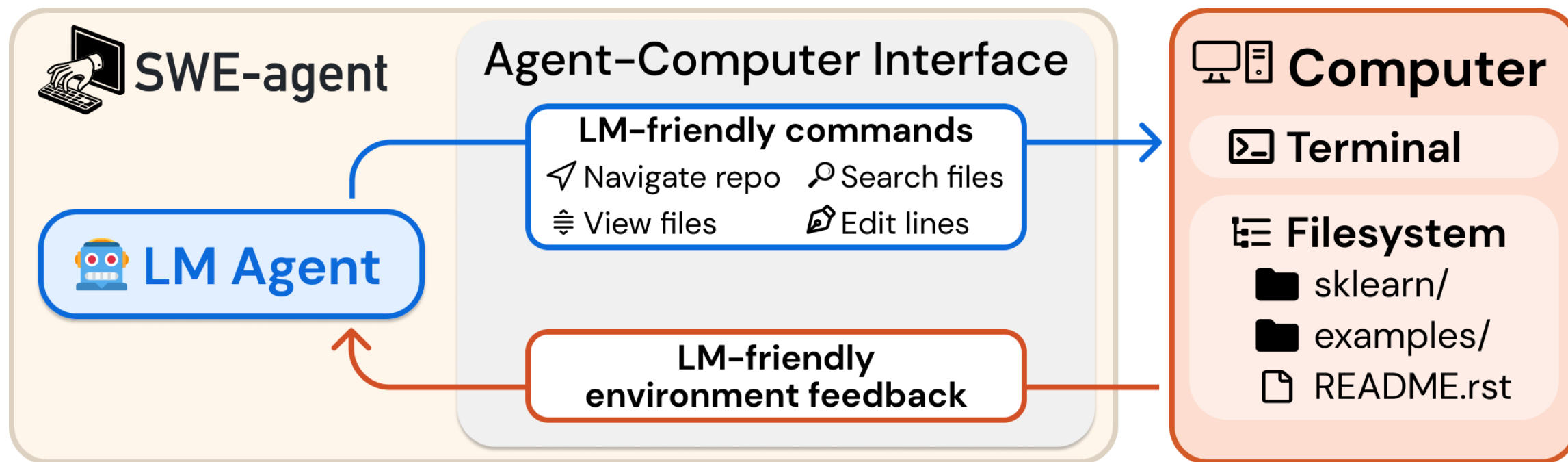
What has the agent actually verified?

Observation	What it supports	What remains open
Edit accepted	Selected syntax checks passed	Behavior of the edited code
Reproduction prints True	Values and shape agree for this zero array	Output dtype, nonzero inputs, other attributes
Agent submits	The controller ended the attempt	Held-out test success

The human PR also checks BitsStored and PlanarConfiguration. This reproduction does not cover those cases.

The interface determines what reaches the model

SWE-agent exposes search, file viewing, editing, and execution through an agent-computer interface.



Our trace shows both sides: useful feedback and a replacement-range error.

A model call inside an execution loop

$$c_t = C_h(q, \tau_t, z_t), \quad u_t \sim \pi_\theta(\cdot \mid c_t)$$

$$a_t = D_h(u_t, z_t), \quad (s_{t+1}, o_{t+1}) \sim E_h(\cdot \mid s_t, a_t)$$

q : task; τ_t : interaction history; z_t : harness state; s_t : environment state.

C_h constructs context. D_h parses tool requests. E_h executes them, including any checks and rollback.

The harness records feedback and updates its state before the next call.

The second edit through the execution loop

Part of the loop	This recorded action
Context C_h	Repair task, earlier feedback, numbered code, current file view
Parser D_h	Parse response u_t into edit request a_t : edit 287:295.
Execution E_h	Write the candidate $\rightarrow$ check $\rightarrow$ restore the backup on failure.
Observation o_{t+1}	Return the syntax error and candidate/original excerpts.

The target file returns to its prior contents. History gains the failed action and its feedback.

The execution loop in mini-SWE-agent

Different implementation: general Bash tool calls, with no automatic edit rollback.

```
while True:
    try:
        check_limits()
        response = model.query(messages)
        messages.append(response)
        outputs = execute(response.tool_calls)
        messages.extend(format_observations(outputs))
    except StopOrRecover as event:
        messages.extend(event.messages)
    if has_exit_message(messages):
        break
```

One response may contain several actions. The pinned version uses tool calls.

Error handling is part of the agent's behavior

Event	Controller behavior	Consequence
Malformed tool response	Corrective feedback, then a retry limit	Repeated format errors can end the run
Command timeout	Terminate it; keep partial output	Partial evidence may guide recovery
Cost limit reached	Check before the next model call	One call can overshoot the limit
Submission signal	End the attempt	Correctness still needs an evaluator

File state and shell state persist differently

Action 1: start at /workspace, MODE unset

```
cd /workspace/project  
export MODE=debug  
echo checkpoint > progress.txt
```

Action 2: new shell with the same defaults

```
pwd  
echo "$MODE"  
cat /workspace/project/progress.txt
```

Action 2 prints /workspace, a blank line, then checkpoint. The file persists across shells.

A shell tool inherits the environment's authority

LocalEnvironment executes commands on the host. A classroom run should use an isolated workspace with explicit access limits.

Repository text and command output can contain instructions. They remain untrusted task data.

Network access, mounted files, credentials, and resource limits determine what a mistaken action can affect.

For our exercise: disposable repository, no host secrets, bounded runtime, controlled network access.

Output truncation changes the evidence

The pinned mini configuration retains the start and end of long output.

```
if len(output) >= 10_000:  
    visible = (output[:5_000]  
               + truncation_notice  
               + output[-5_000:])
```

Units are characters. A failure in the middle may require a targeted command to recover.

Search with general shell commands

No Search

Agent performs manual search with `ls + cd`, or uses `grep` but struggles to interpret the result

💡 First, I should list out all the files in the codebase that may contain the definition of `PVSystem`.

```
$ ls
```

```
benchmarks/  setup.cfg  
docs/        versioneer.py  
pvlib/       README.md  
setup.py     ...
```

💡 It's most likely the source file is in `pvlib/` folder. Let's change directory.

```
$ cd pvlib/
```

Search task: find the file defining class `PVSystem`.

The model chooses shell commands. Here, a directory listing leads to another navigation step.

Which observation would let the model locate `PVSystem` sooner?

Iterative search reveals one match at a time

Iterative Search

Actions to show next/prev search result are repeated many times until results are exhausted

 I will use the `search\_all` command to find occurrences of "PVSystem"...

```
$ search "PVSystem"
```

```
[Result 1/15 (Line 70)]  
[File: pvlib/pvsystem.py]  
22:# a dict of required pa...  
23:_DC_MODEL_PARAMS = {  
24:     'sapm': {  
(97 more lines omitted)
```

 This is not the PVSystem class def...

```
$ next
```

The interface returns a match and its local context.

The model may spend more actions asking for the next result.

How should the interface handle many plausible matches?

Summarized search helps select a file

Summarized Search

- (1) Show all results in single output
- (2) Tell agent to retry if too many results.

 The next step is to locate the `PVSystem` class in the codebase and modify the constructor...

\$ search "class PVSystem"

Found 1 matches for "class
PVSystem" in /pvlib-python:
/pvlib__pvlib-python/pvlib/
pvsystem.py (1 matches)
End of matches

 It's located in `pvlib/pvsystem.py`...

\$ open pvlib/pvsystem.py

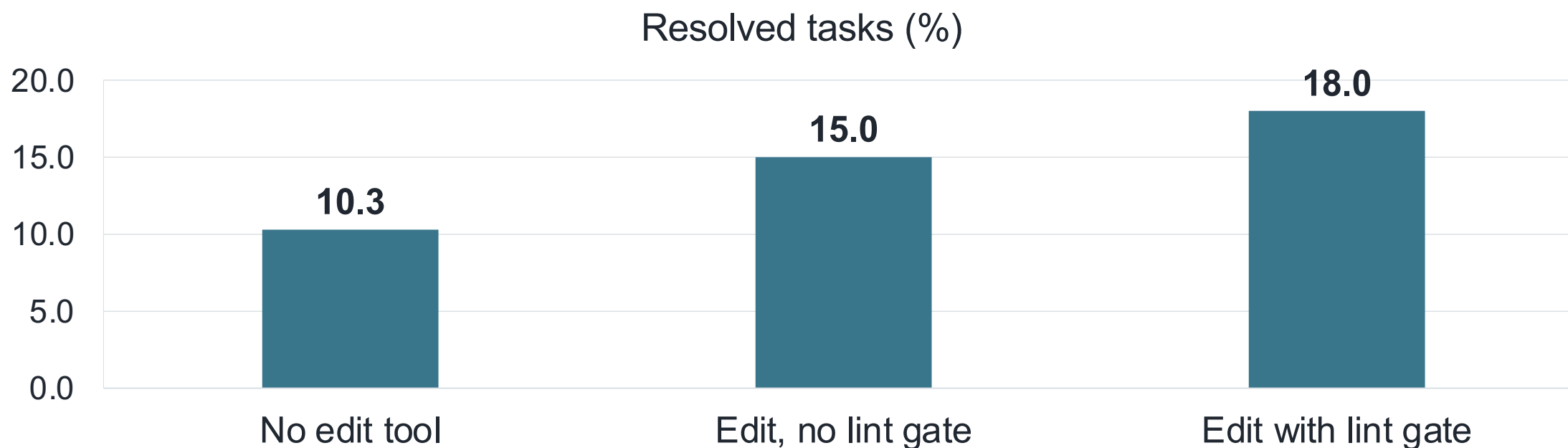
The interface reports a bounded overview before the file is opened.

In the paper, more than 50 results triggers a request to narrow the query.

Would your preferred cutoff change with the model or token budget?

SWE-agent's editing-tool ablation

SWE-bench Lite, 300 tasks, GPT-4 Turbo. Table 3 of SWE-agent.



18.0% versus 15.0% is a 3.0 percentage-point difference in this setting.

The comparison combines lint feedback and enforcement. It does not isolate rollback alone.

Editing and observation choices in the two harnesses

Component	Original SWE-agent	Pinned mini-SWE-agent
Actions	Specialized commands	General Bash tool calls
Editing	Line ranges and a lint gate	No equivalent automatic gate
Observations	Bounded search and file views	Configured output truncation
Model's work	Specialized syntax and exact line ranges	Shell commands and explicit checks

Compare the work each interface asks the model to do, then measure the result.

A lint gate constrains edit order

Remove debug. Assume callers no longer pass it. This checker flags only undefined names.

```
# Initial code
def score(x, debug=False):
    if debug:
        print(x)
    return x
```

```
# First edit removes only the parameter
def score(x):
    if debug:      # undefined until the next edit
        print(x)
    return x
```

Remove the debug block first, or replace the complete function in one edit.

What will lint enforcement change?

Both conditions use the same edit API. Source writes outside that API are blocked.

Condition	Lint feedback	Resulting file state
A: Keep	Same checker and format	Apply the edit and report that state
B: Restore	Same checker and format	Restore the prior file and report that state

Pairs, 4 minutes: predict success and cost for repairs and refactors. Give a concrete case and one essential control.

Matched tasks and measurable outcomes

Use the same model, task snapshots, visible tests, prompts, checker, and diagnostic format. Choose a primary resource budget.

Tune on separate tasks. Freeze both harnesses and define repair/refactor groups before evaluation.

Record final test success, actual cost, latency, and rejected edits. A rejection count alone cannot establish benefit.

Repeat every task in both conditions. Log edit-API bypass attempts and retain unsolved tasks.

Paired effects for repairs and refactors

$$\bar{Y}_i^{(h)} = \frac{1}{R} \sum_{r=1}^R V_i(s_{i,r,T}^{(h)})$$
$$\hat{\Delta} = \frac{1}{N} \sum_{i=1}^N \left(\bar{Y}_i^{(B)} - \bar{Y}_i^{(A)} \right)$$

V_i is the held-out test criterion for task i . R runs per task, N tasks.

Compute Δ separately for repairs and refactors. Positive Δ favors restoring failed edits.

Report task-cluster uncertainty, actual cost, and latency for each group.

A matched probe after one failed edit

Fork the same task, history, files, harness state, and remaining budget before a preselected failing edit.

Apply it under A and B. Count turns to the next nonempty edit that passes lint.

Fix a cap K . Assign K to unfinished probes and report their fraction.

This endpoint can include reversions or whitespace changes. Report those cases and final test success separately.

Stopping status and verified task success

$$\sigma_h(\tau_t, z_t) \in \{\text{continue}, \text{submit}, \text{limit}, \text{error}\}$$

$$V(q, s_T) \in \{0, 1\}$$

σ_h determines whether execution continues. V scores the final state under a specified criterion.

Our pydicom trace reaches submit. Its reproduction passes. Broader test success remains unmeasured here.

What to inspect in a coding-agent harness

The action contract: what the model can request and how the tool changes the repository.

The observation contract: what feedback enters context and what the harness leaves out.

The stopping and evaluation rules: why execution ends and what evidence supports success.

A proposed harness change should name a failure it addresses and a comparison that could refute it.

Training data needs more than a submitted patch

Part of the record	What we can learn from it	Additional check
Failed edit and feedback	How the agent attempts recovery	Did the tool preserve the intended state?
Successful reproduction	A local behavioral check	Does the patch pass independent tests?
Final submitted patch	The proposed code change	Does it satisfy the task without regressions?

Next lecture: constructing executable tasks and deciding which trajectories provide useful supervision.

Readings and code to inspect

SWE-agent, Sections 3–5 and Table 3. Read the interface choices and their ablations.

mini-SWE-agent control-flow guide. Follow `default.py` into the execution environment and configuration.

The pydicom recorded trace. Compare its reproduction, submitted patch, and the human PR.

Exact source revisions, code excerpts, and reuse notices accompany these slides.